



Design and Development of an MQTT- Based Remote HVAC Monitoring System Using ThingSpeak Cloud

EXERCISE NO – 14

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

14. Design and Development of an MQTT-Based Remote HVAC Monitoring System Using ThingSpeak Cloud

1. Project overview:

The proposed **MQTT-Based Remote HVAC Monitoring System** is designed to monitor temperature and humidity conditions remotely using IoT technology. HVAC systems require continuous monitoring to maintain comfortable environmental conditions and improve operational efficiency.

The system uses an **ESP8266 NodeMCU** as the main controller and a **DHT11 sensor** to measure temperature and humidity. The collected sensor data is transmitted through Wi-Fi using the **MQTT communication protocol** and stored on the **ThingSpeak cloud platform**.

An OLED or LCD display is used to show the current temperature and humidity locally. A relay module can be used for basic control of an HVAC device or a simulated load. The system enables remote monitoring, cloud-based data storage, analysis of environmental conditions, and control based on predefined limits.

The system combines wireless communication, cloud technology, and basic automation to provide an efficient HVAC monitoring solution. By using the ESP8266 NodeMCU and MQTT communication, sensor readings can be transmitted remotely with minimal manual intervention. The stored temperature and humidity data can be used to study environmental variations, evaluate HVAC performance, and support better operational and energy-management decisions.

2. Problem Statement:

Conventional HVAC monitoring systems often require manual observation of temperature and humidity conditions. This makes continuous monitoring difficult and may delay the detection of abnormal environmental conditions.

Without remote monitoring and data storage, it is difficult to analyze HVAC performance over time. Excessive temperature or humidity can reduce comfort and may lead to inefficient HVAC operation and unnecessary energy consumption.

Therefore, an IoT-based system is required to continuously monitor HVAC parameters, transmit sensor data remotely, store the readings on a cloud platform, and provide basic control when abnormal conditions are detected.

Furthermore, the lack of real-time remote access makes it difficult for users to monitor HVAC conditions from different locations. Any sudden increase in temperature or humidity may remain unnoticed until manual inspection is performed. By implementing an MQTT-based monitoring system with cloud storage, the HVAC parameters can be continuously observed, recorded, and analyzed remotely, enabling faster response to abnormal conditions and supporting improved system control and energy management.

3. Proposed Solution:

The proposed system uses a **DHT11 sensor** to measure temperature and humidity. The sensor is connected to the **ESP8266 NodeMCU**, which processes the collected data.

The NodeMCU connects to a Wi-Fi network and communicates using the **MQTT protocol**. The sensor readings are transmitted to the cloud and stored on the **ThingSpeak platform** for remote monitoring and analysis.

The current readings are also displayed on an OLED or LCD display. The system compares the measured values with predefined limits. If the temperature exceeds the selected limit, the NodeMCU can activate a relay module to control a connected HVAC device or demonstration load.

The proposed system provides:

- Continuous temperature monitoring
- Humidity monitoring
- MQTT-based communication
- Remote monitoring
- Cloud data storage
- Real-time display
- Basic HVAC control using a relay
- Historical data analysis

4. System Architecture:

The DHT11 sensor continuously measures the temperature and humidity of the monitored HVAC environment. The measured data is sent to the ESP8266 NodeMCU.

The NodeMCU processes the readings and displays them on the OLED or LCD display. Using Wi-Fi, the NodeMCU communicates with an MQTT broker and sends the collected data for cloud monitoring. The data is stored and visualized using the ThingSpeak cloud platform.

The relay module is controlled by the NodeMCU according to the predefined temperature condition. This can be used to demonstrate automatic control of an HVAC load.

System Flow:

DHT11 Sensor → ESP8266 NodeMCU → Wi-Fi/MQTT → ThingSpeak Cloud → Remote User

ESP8266 NodeMCU → Relay Module → HVAC Load/Simulation

This architecture allows the user to monitor HVAC conditions remotely and analyze the stored sensor data

5.Circuit Table

COMPONENT	PIN	CONNECTION
DHT11 Sensor	VCC	3.3 V
DHT11 Sensor	DATA	D4
DHT11 Sensor	GND	GND
OLED Display	SDA	D2
OLED Display	SCL	D1
OLED Display	VCC	3.3 V
OLED Display	GND	GND
Relay	IN	D5
Relay	VCC	5V
Relay	GND	GND

6. Circuit Design:

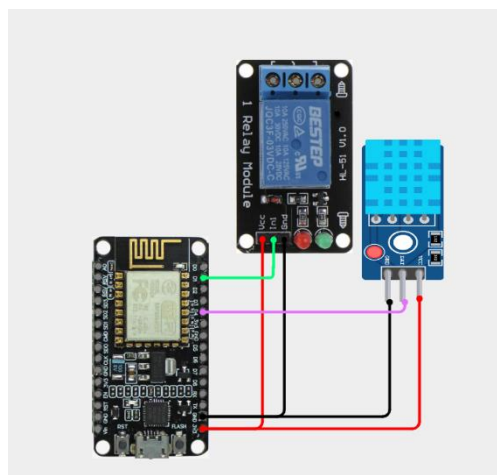
The **DHT11 sensor** is connected to the ESP8266 NodeMCU to measure temperature and humidity. The DATA pin of the DHT11 is connected to digital pin D4.

The OLED display is connected through the I²C communication interface. The SDA pin is connected to D2, while the SCL pin is connected to D1.

The relay input is connected to D5. The relay can be used to control a low-voltage demonstration load or an appropriately isolated HVAC control interface.

The NodeMCU acts as the central controller. It receives data from the sensor, processes the readings, updates the display, communicates through Wi-Fi, and controls the relay based on the programmed conditions.

For simulation, the relay can control a lamp, fan, or virtual load representing the HVAC system.



7. Algorithm:

- Step 1:** Start the system.
- Step 2:** Initialize NodeMCU and connected devices.
- Step 3:** Connect the system to Wi-Fi.
- Step 4:** Establish MQTT communication.
- Step 5:** Read temperature from the DHT11 sensor.
- Step 6:** Read humidity from the DHT11 sensor.
- Step 7:** Display the readings on the OLED/LCD.
- Step 8:** Compare the temperature with the preset limit.
- Step 9:** Control the relay based on the measured condition.
- Step 10:** Publish the sensor data using MQTT.
- Step 11:** Store and display the data on ThingSpeak.
- Step 12:** Repeat the process continuously.

8. Coding:

```
#include <PubSubClient.h>
#include <DHT.h>

const char* WIFI_SSID = "K-APIL";
const char* WIFI_PASSWORD = "kapil.2333";

const char* MQTT_SERVER = "mqtt3.thingspeak.com";

const int MQTT_PORT = 1883;

const char* MQTT_USERNAME = "CDgJBgUtNgIEMyYVBw0zHCs";

const char* MQTT_PASSWORD = "xWeHkKVG2V/e+A4jYKayWLn2";

const char* MQTT_CLIENT_ID = "CDgJBgUtNgIEMyYVBw0zHCs";

const char* CHANNEL_ID = "3460668";
```

```
#define DHT_PIN 2      // GPIO2 = D4
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);


#define RELAY_PIN 5    // GPIO5 = D1

// Most relay modules are ACTIVE LOW
#define RELAY_ON LOW
#define RELAY_OFF HIGH


float HVAC_ON_TEMP = 30.0;

float HVAC_OFF_TEMP = 28.0;


bool hvacStatus = false;

WiFiClient espClient;

PubSubClient mqttClient(espClient);

String mqttTopic;

unsigned long previousMillis = 0;

const unsigned long interval = 20000;

void connectWiFi()
{
  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
```

```
    Serial.print(".");
}

Serial.println();

Serial.println("WiFi Connected!");

Serial.print("IP Address: ");

Serial.println(WiFi.localIP());
}

void connectMQTT()
{
    while (!mqttClient.connected())
    {
        Serial.println();
        Serial.println("Connecting to ThingSpeak MQTT...");

        if (mqttClient.connect(
            MQTT_CLIENT_ID,
            MQTT_USERNAME,
            MQTT_PASSWORD))
        {
            Serial.println("MQTT Connected!");

            Serial.print("MQTT Server: ");
            Serial.println(MQTT_SERVER);

            Serial.print("Channel ID: ");
            Serial.println(CHANNEL_ID);
        }
        else
        {
            Serial.print("MQTT Connection Failed.");

            Serial.print(" Error State: ");

            Serial.println(mqttClient.state());

            Serial.println("Retrying in 5 seconds...");

            delay(5000);
        }
    }
}
```

```
}  
}  
}
```

```
void controlHVAC(float temperature)
```

```
{
```

```
if (temperature >= HVAC_ON_TEMP)
```

```
{
```

```
    hvacStatus = true;
```

```
    digitalWrite(RELAY_PIN, RELAY_ON);
```

```
    Serial.println("Temperature Condition: HIGH");
```

```
    Serial.println("HVAC: ON");
```

```
    Serial.println("Relay: ON");
```

```
}
```

```
else if (temperature <= HVAC_OFF_TEMP)
```

```
{
```

```
    hvacStatus = false;
```

```
    digitalWrite(RELAY_PIN, RELAY_OFF);
```

```
    Serial.println("Temperature Condition: NORMAL");
```

```
    Serial.println("HVAC: OFF");
```

```
    Serial.println("Relay: OFF");
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("Temperature Condition: BETWEEN LIMITS");
```

```
    if (hvacStatus == true)
```



```
{
    digitalWrite(RELAY_PIN, RELAY_ON);

    Serial.println("HVAC: ON");

    Serial.println("Relay: ON");
}
else
{
    digitalWrite(RELAY_PIN, RELAY_OFF);

    Serial.println("HVAC: OFF");

    Serial.println("Relay: OFF");
}
}
```

```
void sendToThingSpeak(float temperature, float humidity)
```

```
{
    int hvacValue = hvacStatus ? 1 : 0;

    int relayValue = hvacStatus ? 1 : 0;
```

```
String payload = "";
```

```
payload += "field1=";
payload += String(temperature, 1);
```

```
payload += "&field2=";
payload += String(humidity, 1);
```

```
payload += "&field3=";
payload += String(hvacValue);
```

```
payload += "&field4=";
payload += String(relayValue);
```

```
Serial.println();
Serial.println("ThingSpeak MQTT Payload:");
```

```
Serial.println(payload);
```

```
String topic = "channels/";
```

```
topic += CHANNEL_ID;
```

```
topic += "/publish";
```

```
Serial.print("MQTT Topic: ");
```

```
Serial.println(topic);
```

```
if (mqttClient.publish(  
    topic.c_str(),  
    payload.c_str()))  
{  
    Serial.println("ThingSpeak: DATA SENT");  
}  
else  
{  
    Serial.println("ThingSpeak: DATA SEND FAILED");  
}  
}
```

```
void setup()  
{  
    Serial.begin(115200);
```

```
    delay(1000);
```

```
    Serial.println();
```

```
    Serial.println(" SMART HVAC MONITORING SYSTEM");
```

```
    Serial.println(" ESP8266 + DHT11 + RELAY");
```

```
    Serial.println(" MQTT + THINGSPEAK");
```

```
dht.begin();
```

```
pinMode(RELAY_PIN, OUTPUT);
```

```
digitalWrite(RELAY_PIN, RELAY_OFF);
```

```
connectWiFi();
```

```
mqttClient.setServer(  
  MQTT_SERVER,  
  MQTT_PORT  
);
```

```
connectMQTT();
```

```
Serial.println();
```

```
Serial.println("System Ready!");
```

```
Serial.println();  
}
```

```
void loop()  
{
```

```
  if (WiFi.status() != WL_CONNECTED)  
  {  
    connectWiFi();  
  }
```

```
  if (!mqttClient.connected())  
  {  
    connectMQTT();  
  }
```

```
mqttClient.loop();
```

```
unsigned long currentMillis = millis();
```

```
if (currentMillis - previousMillis >= interval)
{
    previousMillis = currentMillis;
```

```
    float temperature = dht.readTemperature();
```

```
    float humidity = dht.readHumidity();
```

```
    if (isnan(temperature) || isnan(humidity))
    {
        Serial.println();
```

```
        Serial.println("DHT11 SENSOR ERROR!");
```

```
        Serial.println("Check DHT11 wiring.");
```

```
        return;
    }
```

```
    if (temperature < 0.0 ||
        temperature > 50.0 ||
        humidity < 20.0 ||
        humidity > 90.0)
    {
        Serial.println();
```

```
        Serial.println("INVALID DHT11 READING!");
```

```
        Serial.print("Temperature: ");
```

```
Serial.println(temperature);
```

```
Serial.print("Humidity: ");
```

```
Serial.println(humidity);
```

```
Serial.println("HVAC CONTROL SKIPPED");
```

```
return;
```

```
}
```

```
Serial.println();
```

```
Serial.print("Temperature: ");
```

```
Serial.print(temperature, 1);
```

```
Serial.println(" °C");
```

```
Serial.print("Humidity: ");
```

```
Serial.print(humidity, 1);
```

```
Serial.println(" %");
```

```
controlHVAC(temperature);
```

```
sendToThingSpeak(
```

```
    temperature,
```

```
    humidity
```

```
);
```

```
}
```

```
}
```

9. System Working:

When the system is powered ON, the ESP8266 NodeMCU initializes the DHT11 sensor, display, relay module, and Wi-Fi connection.

The DHT11 continuously measures the environmental temperature and humidity. The NodeMCU receives these readings and displays them on the OLED or LCD screen.

For example:

Temperature: 25 °C

Humidity: 55 %

Status: Normal

The NodeMCU then publishes the sensor data using the MQTT communication protocol. The data is made available for remote monitoring and stored on the ThingSpeak cloud platform.

The user can access the cloud platform to observe the current readings and historical changes in temperature and humidity.

If the temperature exceeds the selected threshold, the relay can be activated to represent HVAC control. This demonstrates how the system can support automatic environmental control in addition to monitoring.

```
-----  
Temperature: 32.0 °C  
Humidity: 85.4 %  
Temperature Condition: HIGH  
HVAC: ON  
Relay: ON
```

```
ThingSpeak MQTT Payload:  
field1=32.0&field2=85.4&field3=1&field4=1  
MQTT Topic: channels/3460668/publish  
ThingSpeak: DATA SENT  
-----
```

```
-----  
Temperature: 32.8 °C  
Humidity: 88.2 %  
Temperature Condition: HIGH  
HVAC: ON  
Relay: ON
```

```
ThingSpeak MQTT Payload:  
field1=32.8&field2=88.2&field3=1&field4=1  
MQTT Topic: channels/3460668/publish  
ThingSpeak: DATA SENT  
-----
```

```
-----  
Temperature: 33.3 °C  
Humidity: 89.7 %  
Temperature Condition: HIGH  
HVAC: ON  
Relay: ON
```

```
ThingSpeak MQTT Payload:  
field1=33.3&field2=89.7&field3=1&field4=1  
MQTT Topic: channels/3460668/publish  
ThingSpeak: DATA SENT  
-----
```

```
-----  
Temperature: 29.3 °C  
Humidity: 69.3 %  
Temperature Condition: BETWEEN LIMITS  
HVAC: ON  
Relay: ON
```

```
ThingSpeak MQTT Payload:  
field1=29.3&field2=69.3&field3=1&field4=1  
MQTT Topic: channels/3460668/publish  
ThingSpeak: DATA SENT  
-----
```

```
-----  
Temperature: 29.1 °C  
Humidity: 69.6 %  
Temperature Condition: BETWEEN LIMITS  
HVAC: ON  
Relay: ON
```

```
ThingSpeak MQTT Payload:  
field1=29.1&field2=69.6&field3=1&field4=1  
MQTT Topic: channels/3460668/publish  
ThingSpeak: DATA SENT  
-----
```

```
-----  
Temperature: 28.9 °C  
Humidity: 70.0 %  
Temperature Condition: BETWEEN LIMITS  
HVAC: ON  
Relay: ON
```

My Channels

New Channel

Search by tag

Q

Name	Created	Updated
🔒 HVAC Private Public Settings Sharing API Keys Data Import / Export	2026-08-19	2026-08-19 05:37

MQTT Devices

Add a new device

Device Details:

ESP8266 HVAC

No description

Authorized Channels and Permissions:

HVAC (3460668)

✓ publish

✓ subscribe

MQTT Client ID:

CDgJBgUtNgIEMyYVBw0zHCs

Edit

Delete

ThingSpeak™

Channels

Apps

Devices

Support

HVAC

Temperature

33.0

32.0

31.0

30.0

29.0

28.0

11:02:00

11:03:00

11:04:00

11:05:00

Date

ThingSpeak.com

HVAC

Humidity

90.0

85.0

80.0

75.0

70.0

65.0

11:02:00

11:03:00

11:04:00

11:05:00

Date

ThingSpeak.com

Field 3 Chart

HVAC

HVAC Status

1.00

0.80

0.60

0.40

0.20

0.00

11:02:00

11:03:00

11:04:00

11:05:00

Date

ThingSpeak.com

Field 4 Chart

HVAC

Relay Status

1.00

0.80

0.60

0.40

0.20

0.00

11:02:00

11:03:00

11:04:00

11:05:00

Date

ThingSpeak.com

10. Bill of Materials:

COMPONENT	QUANTITY
ESP8266 NodeMCU	1
DHT11 Sensor	1
Relay Module	1
OLED/LCD Display	1
Buzzer	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:

The prototype can be implemented in the following stages.

Stage 1 – Sensor Connection

Connect the **DHT11 sensor** to the **ESP8266 NodeMCU** and verify the temperature and humidity readings using the Arduino IDE Serial Monitor.

Stage 2 – Local Display

Connect an **OLED/LCD display** to show the current HVAC parameters.

Example:

HVAC MONITOR

Temperature: 25 °C

Humidity: 55 %

Status: NORMAL

Stage 3 – Wi-Fi Connection

Program the ESP8266 NodeMCU to connect to a Wi-Fi network for remote communication.

Stage 4 – MQTT and Cloud Connection

Configure the NodeMCU to publish the sensor readings using the **MQTT protocol** and connect the system to the **ThingSpeak cloud platform**.

The cloud dashboard can contain:

- Temperature field
- Humidity field
- Temperature graph
- Humidity graph
- Historical data
- System status

Stage 5 – Relay Control

Set a temperature threshold for HVAC control.

Example:

Temperature $> 26\text{ }^{\circ}\text{C}$ $\rightarrow$ Relay ON

Temperature $\leq 26\text{ }^{\circ}\text{C}$ $\rightarrow$ Relay OFF

The relay can be connected to a simulated load to represent the operation of an HVAC device.

Stage 6 – Prototype Setup

Arrange the **NodeMCU, DHT11 sensor, relay module, and OLED/LCD display** on the breadboard or inside a suitable enclosure.

The DHT11 sensor should be placed at an appropriate location where it can properly measure the temperature and humidity of the HVAC environment

12.Results & Performance Analysis:

The developed MQTT-Based Remote HVAC Monitoring System successfully monitors temperature and humidity using the DHT11 sensor. The ESP8266 NodeMCU processes the sensor data and sends it remotely through MQTT to the ThingSpeak cloud platform.

The system performance can be summarized as follows:

- Continuous monitoring of temperature and humidity.
- Real-time display of sensor readings.
- Remote data transmission using MQTT.
- Cloud storage and graphical analysis using ThingSpeak.
- Relay control based on preset temperature conditions.
- Supports HVAC performance analysis and energy management.

Overall, the system provides a simple, low-cost, and effective solution for remote HVAC monitoring and control.